

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

11. Monitorización I: Introducción y Métricas	1
11.1. ¿Qué es la monitorización?	1
11.1.1. Objetivos de la monitorización	2
11.1.2. Sistema Simple de Monitorización	3
11.1.3. Sistema de Monitorización en Hadoop	4
11.2. Métricas	6
11.2.1. Tipos de Métricas	6
11.2.2. Características de los Datos	8
11.3. Métricas de Hadoop	8
11.3.1. HDFS	8
11.3.2. MapReduce	12
11.3.3. YARN	14

Listado de acrónimos

HDFS	Hadoop Distributed File System
JVM	Java Virtual Machine
YARN	Yet Another Resource Negotiator

Capítulo 11

Monitorización I: Introducción y Métricas

11.1. ¿Qué es la monitorización?

La monitorización es uno de los aspectos más importantes ya que permite medir y garantizar la disponibilidad. Un sistema sin monitorización no garantiza lo que estas proporcionado. Por ello es necesario controlar de manera automática que los sistemas estén funcionando y puedan reaccionar de manera rápida ante los fallos. La monitorización no solo mejora la disponibilidad, también proporciona información a nivel operativo, permitiendo realizar estimaciones de tiempo y recursos. Estos son los motivos por lo que es necesario monitorizar todos los aspectos de clúster. Desde el entorno físico donde se encuentra, hasta los servicios de los que depende.

Los sistemas básicos que se deben monitorizar son:

- **Datos en tiempo real y el histórico:** Es necesario que se pueda visualizar de forma gráfica la información relevante del sistema, además se debe de disponer de un histórico donde se almacenen los datos y se puedan consultar
- **Gestión de eventos y alarmas:** Se deben definir eventos como un uso excesivo de una CPU o una bajada en la velocidad de la conexión.

La medición del rendimiento de un sistema es una actividad clave en el desarrollo y mantenimiento de este. Con la infraestructura hardware y software adecuada, se es capaces de recoger información en tiempo real de parámetros tales como: temperatura (ambiental y de los nodos), humedad, utilización de recursos (red, disco, memoria), etc.

Los tipos de monitorización son:

- **Activos:** Desde un sistema de monitorización se comprueba el estado y métricas de los dispositivos.
- **Pasivos:** El sistema que está siendo monitorizado envía una alarma cuando sucede un evento que debe ser atendido

11.1.1. Objetivos de la monitorización

Los objetivos más importantes para hacer uso de la monitorización son:

- **Correlación entre distintas fuentes de datos:** La monitorización de sistemas Big Data puede cubrir un amplio rango de fuentes de datos potenciales, aquí se pueden incluir gran cantidad de componentes de los sistemas con una gran variedad de software para la computación de nodos, switches, archivos de sistema... Los datos recopilados a partir de estas fuentes deben ser correlacionados para poder proporcionar una visión del comportamiento del sistema. Esto requiere la captura de una gran cantidad de datos de fuentes muy dispares, con comportamientos distintos y con interfaces muy variadas.
- **Gestión de los datos y análisis en el tiempo:** Por lo general las plataformas Big Data tienen a durar múltiples años, y la búsqueda de tendencia en su comportamiento puede ser de gran importancia para entender y prevenir su comportamiento en el tiempo. Debido a la gran cantidad de datos de monitorización que se producen en estas plataformas, la escala, gestión y almacenamiento de esta información en un factor de gran importancia para realizar estos análisis de tendencia.
- **Captura de información relevante:** Después de realizar los análisis de los datos y correlación, se puede determinar que ciertos resultados proporcionan información más relevante sobre el estado y la salud de un clúster. Estos tipos de análisis permitirán reducir los requisitos de almacenamiento de datos, debido a la eliminación de reportes poco relevantes.

En arquitecturas Big Data la complejidad crece exponencialmente, así como el coste de los equipos, y su mantenimiento. Debido a esto existen distintos objetivos secundarios de la monitorización relacionados con el coste económico del clúster.

- **Utilización de recursos y optimización del reparto de carga:** Para ello es necesario obtener información efectiva de los recursos críticos constantes para distintos tipos de trabajo.
- **Predecir la tolerancia a fallos y predecir fallos de hardware:** Normalmente debido a la gran cantidad de componentes en los clústeres el tiempo medio entre fallos o Mean Time Between Failures (MTBF!) disminuye con rapidez. Algunas aplicaciones del mundo real como los estudios de la dinámica de moléculas pueden durar desde algunas horas hasta varios días para

completar su ejecución. Debido a esto es normal que surjan distintos fallos en una ejecución, la toma de datos precisos para la detección y previsión de estos fallos puede ayudar a mejorar el sistema en la ejecución de futuras aplicaciones.

- **Mejorar la eficiencia energética:** El consumo energético es uno de los factores clave a la hora de diseñar un sistema clúster, así como el desarrollo de software que cumpla requisitos de energía específicos. Una de las recomendaciones para mejorar la eficiencia es tener poca diversidad de interfaces de medición de energía y poder de distintos vendedores, ya que estas pueden diferir entre vendedores, y podría afectar a los procesos de medición.
- **Notificación de anomalías y errores:** Mediante las alarmas se puede avisar al usuario de que algo no funciona correctamente o está a punto de llegar a ser un fallo
- **Conocimiento del rendimiento:** Debido a la complejidad de los sistemas, es necesario poder comprobar el estado actual del sistema y si el rendimiento es el esperado.
- **Mejorar la gestión de logs:** Los logs son útiles para encontrar errores y fallos, si se dispone de un sistema centralizado de logs es más sencillo ver por qué ha fallado el sistema.
- **Optimización de sistemas:** Con los datos generados de monitorizar un sistema se pueden realizar optimizaciones futuras del sistema
- **Detectar el origen de los incidentes:** La monitorización de los sistemas permite detectar cual componente del sistema falla
- **Planificación de crecimiento:** Mediante el uso de los sistemas se puede detectar los elementos que deben mejorarse

11.1.2. Sistema Simple de Monitorización

Un sistema de monitorización sencillo necesita cuatro componentes clave: un servidor o proceso coordinador, conexiones para sondear los *hosts* del sistema distribuido y recopilar la información necesaria, un repositorio para almacenar la información recopilada y una interfaz gráfica de usuario como front-end [Lak14].

El servidor de monitorización consolida la entrada recibida mediante el sondeo de los *hosts* del sistema distribuido y escribe la salida detallada (así como resumida) en un repositorio. Una consola proporciona opciones de visualización para los datos que se pueden resumir utilizando varios parámetros, como el evento de monitorización, el servidor, el tipo de alerta, etc.

Una arquitectura de sistema de monitorización simple como ésta no es escalable. Si hubiera que monitorizar miles de *hosts*, el servidor de monitorización tendría que gestionar mil conexiones, procesar y consolidar la salida, y presentarla en la consola en unos pocos segundos. Con cada *host* que se añada al sistema, la carga

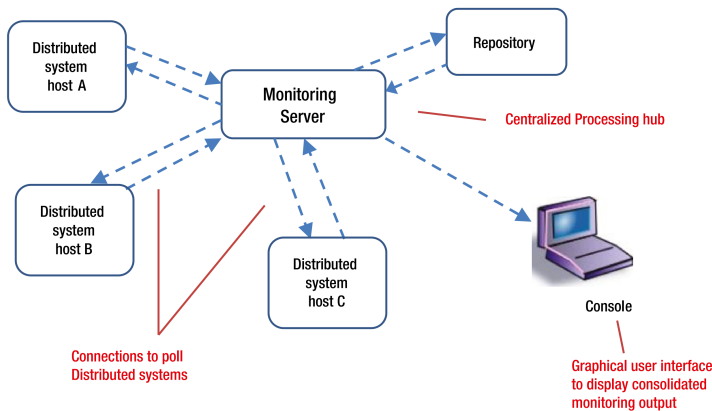


Figura 11.1: Arquitectura simple de monitorización

del servidor de monitorización aumentará. Después de un cierto número de *hosts*, no podrá añadir más, porque el servidor simplemente no será capaz de soportarlos. Además, el gran volumen de sondeos aumentará el tráfico de la red y afectará al rendimiento general del sistema. A esto hay que añadir las complejidades de un cluster Hadoop, en el que hay que tener en cuenta el papel de un nodo mientras se consolidan los datos para él, así como resumir los datos para múltiples nodos con el mismo rol.

11.1.3. Sistema de Monitorización en Hadoop

Un sistema de monitorización simple sigue la misma disposición de procesamiento que el diseño tradicional cliente-servidor: un único servidor de monitorización centralizado realiza todo el procesamiento, y a medida que el número de *hosts* aumenta, también lo hace la carga de procesamiento. El tráfico de red también es importante a nivel de carga, ya que los datos sondeados de los *hosts* se consolidan en el servidor de monitorización [Lak14].

Al igual que la arquitectura distribuida de Hadoop supone una notable mejora de la eficiencia con respecto al procesamiento tradicional cliente-servidor, un modelo de procesamiento distribuido puede mejorar un sistema de monitorización simple. En Hadoop ya no hay un servidor centralizado que se convierta en un cuello de botella de procesamiento o en un punto único de fallo. Cada nodo es un participante activo realizando parte del procesamiento en paralelo. Cada uno de estos procesos localizados puede transmitir datos a otros nodos en el clúster y también recibir copias de datos de otros nodos en el clúster. Un proceso de sondeo puede

obtener los datos de monitorización de todo el clúster desde cualquiera de los nodos del clúster con una frecuencia predeterminada. Los datos pueden ser escribirse en un repositorio y almacenarse para su posterior procesamiento o mostrarse mediante un *frontend* gráfico o basado en la web.

Con esta arquitectura, incluso añadiendo 1000 hosts el rendimiento del proceso de monitorización/supervisión no se vería afectado. Por ejemplo, el proceso de sondeo puede seguir sondeando desde cualquiera de los nodos y no tiene que hacer múltiples conexiones. Por otro lado, los nodos del clúster transmiten datos a un canal común que es recibido por todos los demás nodos, por lo que la sobrecarga sería inapreciable.

Por lo tanto, el aumento del número de nodos no afecta al proceso de sondeo ni al rendimiento del sistema lo que hace que la arquitectura sea altamente escalable. En comparación con los sistemas de monitorización tradicionales el único trabajo extra que hay que hacer es aplicar la configuración del proceso de monitorización a todos los nodos.

En resumen, los procesos de monitorización en los nodos individuales calculan "datos de monitorización locales". Los datos de monitorización tienen que ser calculados localmente; porque Hadoop es un sistema distribuido de múltiples nodos en el que los datos se distribuyen en sus numerosos DataNodes y según la filosofía de Hadoop de "llevar el procesamiento a datos", los datos se procesan localmente, es decir, donde residen, en los DataNodes.

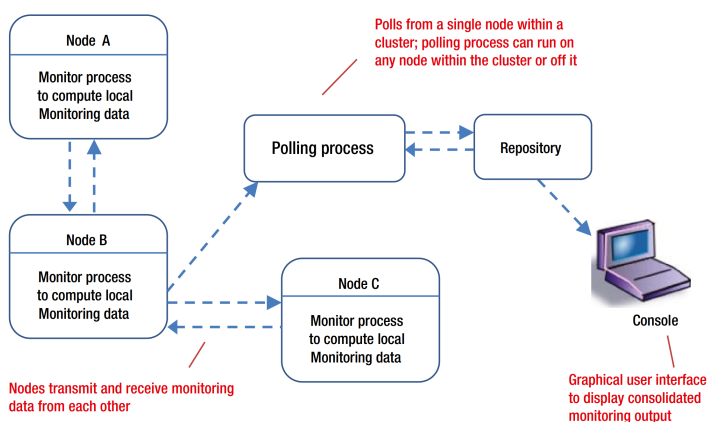


Figura 11.2: Arquitectura simple de monitorización

11.2. Métricas

Las métricas capturan un valor perteneciente al sistemas en un punto específico en el tiempo - por ejemplo, el número de usuarios conectados a una aplicación web. Por lo tanto, las métricas suelen recogerse una vez por segundo, una por minuto o en otro intervalo regular para supervisar un sistema a lo largo del tiempo [WSV14].

11.2.1. Tipos de Métricas

Hay dos categorías importantes de métricas: métricas de trabajo y métricas de recursos. Es necesario que para cada componente de la infraestructura determinar qué métricas de cada uno de los tipos están disponibles y luego establecer los procedimientos necesarios de recolección.

Métricas de Trabajo

Las métricas de trabajo indican el estado de salud del sistema midiendo su rendimiento útil. Se suelen desglosar en cuatro subtipos:

- **throughput**: La productividad es la cantidad de trabajo que el sistema realiza por unidad de tiempo. Suele registrarse como un número absoluto.
- **success**: Las métricas de éxito representan el porcentaje de trabajo que se ha ejecutado con éxito.
- **error**: Las métricas de error capturan el número de resultados erróneos, normalmente expresados como una tasa de errores por unidad de tiempo o normalizados por el rendimiento para obtener errores por unidad de trabajo. Las métricas de error a menudo se capturan por separado de las métricas de éxito cuando hay varias fuentes potenciales de error, algunas de las cuales son más graves o procesables que otras.
- **performance**: Las métricas de rendimiento cuantifican la eficiencia con la que un componente realiza su trabajo. La métrica de rendimiento más común es la latencia, que representa el tiempo necesario para completar una unidad de trabajo. La latencia puede expresarse como una media o como un percentil, como .^{el} 99 % de las solicitudes se devuelven en 0, [YSC⁺18]

Estas métricas son muy importantes para la observabilidad del sistema dado que permiten responder rápidamente a preguntas como: ¿está el sistema disponible y haciendo activamente aquello para lo que fue construido? ¿Con qué rapidez está produciendo trabajo? ¿Cuál es la calidad de ese trabajo?.

Métricas de Recursos

La mayoría de los componentes de la infraestructura de software sirven como recursos. Algunos recursos son de bajo nivel; por ejemplo, la CPU, la memoria, los discos y las interfaces de red. Pero un componente de nivel superior, como una base de datos también puede considerarse un recurso si otro sistema los necesita.

Las métricas de recursos deben a ayudar a contar con una imagen detallada del estado de un sistema, y por ello, son muy valiosas para el análisis y el diagnóstico de problemas. Las métricas a recopilar de cada recursos se pueden organizar en cuatro áreas clave:

- **utilization:** La utilización es el porcentaje de tiempo que el recurso está ocupado, o el porcentaje de la capacidad del recurso que está en uso.
- **saturation:** La saturación es una medida de la cantidad de trabajo solicitado que el recurso aún no puede atender, a menudo en cola.
- **errors:** Los errores representan errores internos que pueden no ser observables en el trabajo que produce el recurso.
- **availability:** La disponibilidad representa el porcentaje de tiempo que el recurso ha respondido a las solicitudes. Esta métrica sólo está bien definida para los recursos cuya disponibilidad puede comprobarse de forma activa y regular.

Eventos

Además de las métricas, que se recogen de forma más o menos continua, algunos sistemas de monitorización también pueden capturar eventos: **sucesos discretos y poco frecuentes** que pueden proporcionar un contexto crucial para entender qué ha cambiado en el comportamiento del sistema.

Algunos ejemplos de eventos relevantes son los siguientes:

- **Cambios:** Lanzamientos de código interno, construcciones y fallos de construcción
- **Alertas:** Alertas generadas internamente o notificaciones de terceros
- **Escalado:** Adición o eliminación de hosts.

Un evento suele contener suficiente información para que pueda interpretarse por sí mismo, a diferencia de un punto de datos de una métrica, que generalmente sólo tiene sentido en su contexto. Los eventos capturan lo que ha ocurrido, en un momento dado, con información adicional opcional.

Los eventos se utilizan a veces para generar alertas, por ejemplo, alguien debe ser notificado cuando un trabajo crítico ha fallado. A su vez, también se utilizan para analizar problemas y correlacionarlos entre sistemas. En general son datos valiosos que deben recogerse siempre que sea posible.

11.2.2. Características de los Datos

Los datos recogidos por un sistema de monitorización deben de tener las siguientes características:

- **Entendibles:** Se debe poder determinar rápidamente cómo se capturó cada métrica o evento y qué representa.
- **Frecuentes:** Si se recogen métricas con muy poca frecuencia o se promedian los valores durante largos periodos de tiempo, se puede perder la capacidad de reconstruir con precisión el comportamiento del sistema. Por ejemplo, los periodos de utilización del 100 % de los recursos quedarán ocultos si se promedian con periodos de menor utilización. Por lo tanto, se deberán recoger las métricas de cada sistema con una frecuencia que no oculte los problemas, sin recogerlas con tanta frecuencia que la monitorización se convierta en una carga perceptible para el sistema (efecto observador) o cree ruido en los datos de la monitorización mediante el muestreo de intervalos de tiempo que son demasiado cortos para contener datos significativos.
- **Larga duración:** Es necesario conservar los datos *“en bruto” (raw)* sin agregar durante un largo periodo de tiempo para así disponer de material suficiente para alimentar a los sistemas de análisis automático. Conservar los datos sin procesar durante un periodo de tiempo significativo hace que sea mucho más fácil saber qué es lo “normal”, especialmente si las métricas tienen variaciones mensuales, estacionales o anuales.

11.3. Métricas de Hadoop

Desde el punto de vista de las operaciones, los clústeres de Hadoop son increíblemente resistentes a los fallos del sistema. El diseño de Hadoop es el adecuado para tener una eficiente tolerancia a fallas y gestionar la caída de racks enteros.

Cuando funciona correctamente, un clúster de Hadoop puede manejar una cantidad realmente masiva de datos: hay muchos clústeres de producción que gestionan petabytes de datos cada uno. La supervisión de cada uno de los subcomponentes de Hadoop es esencial para mantener los trabajos en marcha y el clúster en funcionamiento [Sin15].

Las métricas de Hadoop pueden dividirse en las siguientes categorías: HDFS, MapReduce, YARN.

11.3.1. HDFS

Los DataNodes se comunican con su NameNode (y viceversa) a través de un heartbeat periódico¹. El heartbeat contiene (entre otras cosas) información sobre la salud del DataNode. Es a través de este mecanismo que el NameNode es capaz de conocer el estado de todos los DataNodes en el cluster.

¹por defecto cada tres segundos, establecido por `dfs.heartbeat.interval`

El fallo del NameNode sin ninguna réplica en espera provocará que los datos almacenados en el clúster no estén disponibles. Por lo tanto, la supervisión del NameNode primario y de su NameNode secundario/de reserva es fundamental para garantizar una alta disponibilidad del clúster. Además de informar de las métricas sobre sí mismos, los NameNodes también pueden informar de las métricas sobre todos los DataNodes, proporcionando una buena visión del resto de su despliegue HDFS.

Métricas emitidas del NameNode

- **CapacityRemaining** representa la capacidad total disponible que queda en todo el cluster de HDFS, lo cual puede resultar en quedarse sin espacio en disco en HDFS y provocar un colapso en el sistema. Los DataNodes que se queden sin espacio fallarán al arrancar y no se podrá escribir en los DataNodes en ejecución. Además, cualquier trabajo en ejecución que escriba datos temporales puede fallar debido a la falta de espacio. Afortunadamente, HDFS facilita o bien la creación de nuevos DataNodes para añadir capacidad o bien añadir discos adicionales a los DataNodes existentes.



Es una buena práctica asegurarse de que el uso del disco nunca supere el 80 por ciento de su capacidad.

- **MissingBlocks**: Los bloques corruptos o faltantes pueden indicar un clúster en condiciones precarias de funcionamiento. Normalmente, al recibir una solicitud de lectura, un DataNode localiza el bloque solicitado y lo devuelve al cliente, junto con una suma de comprobación(checksum) de los datos, calculada y almacenada en el momento de escribir el bloque en el DataNode. Si la suma de comprobación de los datos recibidos no coincide con la suma de comprobación almacenada, el cliente informa al NameNode de la corrupción y lee los datos de una de las otras réplicas. El NameNode, por su parte, programa una réplica del bloque desde una de las copias viables, y luego le ordena al DataNode original que elimine su copia corrupta. Un bloque perdido es mucho peor que un bloque corrupto, porque un bloque perdido no se puede recuperar copiando una réplica. Un bloque perdido representa un bloque del que no existe ninguna copia conocida en el clúster.
- **VolumeFailuresTotal**: HDFS permite que los discos fallen sin afectar a las operaciones del DataNode, hasta que se alcance un valor umbral. Por defecto, este valor es 0 lo que significa que cualquier fallo de volumen apagará el DataNode. En un clúster de producción donde los DataNodes suelen tener 6, 8 o 12 discos se establece este parámetro a 1 o a 2. Aunque un volumen fallido no hará que el cluster se detenga, lo más probable es sea necesario

conocer fallos de hardware, aunque sólo sea para poder reemplazar el hardware erróneo. Dado que HDFS fue diseñado teniendo en cuenta los fallos de hardware, con una replicación por defecto de tres, un volumen que falle nunca debería ser señal de pérdida de datos.

- **FilesTotal:** es un recuento continuo del número de archivos que están siendo rastreados por el NameNode. Dado que el NameNode almacena todos los metadatos en memoria, cuanto mayor sea el número de archivos (o bloques, o directorios) rastreados, más memoria necesitará el NameNode. La regla general para los requisitos de memoria de NameNode es que cada objeto (archivo, directorio, bloque) rastreado por NameNode consume aproximadamente 150 bytes de memoria. Si cada archivo se asigna a un bloque (los archivos suelen abarcar varios bloques), esto supone aproximadamente 300 bytes por archivo, 900 con un factor de replicación de 3.
- **TotalLoad** es el número actual de accesos concurrentes a archivos (lectura/escritura) en todos los DataNodes. Esta métrica no suele ser indicativa de un problema, aunque puede poner de manifiesto cuestiones relacionadas con la ejecución del trabajo. Dado que los nodos *Workers* que ejecutan el *daemon* DataNode también realizan tareas de MapReduce, los periodos prolongados de alta E/S (indicados por un alto TotalLoad) generalmente se traducen en un rendimiento degradado de la ejecución del trabajo. El seguimiento de TotalLoad a lo largo del tiempo puede ayudar a identificar problemas de rendimiento en la ejecución de los trabajos.
- **BlockCapacity/BlocksTotal:** Al igual que FilesTotal, vigilar el número total de bloques en el clúster es esencial para su funcionamiento continuo. La capacidad de bloques y la capacidad de archivos están estrechamente relacionadas, pero no son lo mismo (los archivos pueden abarcar varios bloques); es necesario supervisar tanto la capacidad de bloques como la de archivos para garantizar un funcionamiento ininterrumpido.
- **UnderReplicatedBlocks** es el número de bloques con replicación insuficiente. El factor de replicación de Hadoop es configurable por cliente o por archivo. El factor de replicación por defecto es tres, lo que significa que cada bloque se almacenará en tres DataNodes. Si se presentase un pico grande y repentino en el número de bloques insuficientemente replicados, es probable que un DataNode haya muerto.
- **NumStaleDataNodes:** Cada tres segundos (por defecto), los DataNodes envían un mensaje de heartbeat al NameNode. Un DataNode puede no emitir un heartbeat por una serie de razones: los problemas de red o la alta carga son algunas de las causas más comunes. Un DataNode que no emite un *heartbeat* debido a la alta carga del sistema (o a una red lenta) probablemente hará que cualquier lectura o escritura en ese DataNode se retrase también. Un aumento repentino del número de DataNodes obsoletos puede significar un incremento posterior de nodos muertos.

Tabla 11.1: Métricas JVM del NameNode

Nombre	Descripción
ConcurrentMarkSweep count	Número de colecciones de la anterior generación
ConcurrentMarkSweep count	Tiempo transcurrido de las colecciones de la anterior generación, en milisegundos

Métricas JVM del NameNode

Dado que NameNode se ejecuta en la máquina virtual de Java (JVM), depende de los procesos de recolección de basura de Java para liberar memoria. Cuanta más actividad haya en tu cluster HDFS, más a menudo se ejecutará la recolección de basura. La recolección de basura es uno de los procesos más costosos dentro de la operativa de la JVM. Si se detectan pausas excesivas durante la recolección de basura, puede considerar la posibilidad de actualizar la versión del JDK o el recolector de basura. Además, puede ajustar su tiempo de ejecución de Java para minimizar la recolección de basura.

Dado que un NameNode suele estar alojado en una máquina con amplios recursos computacionales, se recomienda utilizar el recolector de basura CMS (ConcurrentMarkSweep), especialmente porque los cuellos de botella en el NameNode provocarán una degradación del rendimiento en todo el clúster.



El recolector CMS debería utilizarse para aplicaciones que requieren bajos tiempos de pausa y que pueden compartir recursos con el recolector de basura. Algunos ejemplos son las aplicaciones de interfaz de usuario de escritorio que responden a eventos, un servidor web que responde a una solicitud o una base de datos que responde a consultas.

Las colecciones *ConcurrentMarkSweep* liberan la memoria no utilizada en la antigua generación del *heap*. Esto proporciona una recolección de basura de baja pausa, lo que significa que aunque detiene temporalmente los hilos de la aplicación, sólo lo hace de forma intermitente.

Si el CMS tarda más de unos segundos en completarse, o se produce con mayor frecuencia, es posible que tu NameNode no tenga suficiente memoria asignada a la JVM para funcionar de forma eficiente

Métricas del DataNode

Las métricas de DataNode son métricas a nivel de host específicas de un DataNode en particular.

- **Remaining:** Un único DataNode que se quede sin espacio podría provocar rápidamente fallos en todo el clúster, ya que los datos se escriben en un grupo cada vez más reducido de DataNodes disponibles. El seguimiento de esta métrica a lo largo del tiempo es esencial para mantener un clúster en un funcionamiento óptimo.



Será necesario alertar sobre la capacidad restante del DataNode caiga a menos del 10 por ciento

- **NumFailedVolumes:** Por defecto, un solo volumen que falle en un DataNode hará que todo el nodo se desconecte. Cuando un DataNode se desconecta, el NameNode debe copiar cualquier bloque infra-replicado que se haya perdido en ese nodo, causando un incremento grande de tráfico de red y una potencial degradación del rendimiento. Para evitar una avalancha de actividad de red por el fallo de un solo disco se debe establecer el nivel tolerado de fallo de volumen en la configuración de a un nivel aceptable (N) tal y como se indica en la siguiente propiedad:

```
<property>

    <name>dfs.datanode.failed.volumes.tolerated</name>

    <value>N</value>

</property>
```

11.3.2. MapReduce

El marco de monitorización de MapReduce se basa en una serie de métricas de conteo que permiten rastrear las estadísticas de la ejecución de trabajos de MapReduce. Los contadores son un mecanismo que permite ver lo que realmente está sucediendo durante la ejecución de un trabajo MapReduce proporcionando un recuento del número de veces que ha ocurrido un evento. Esta información será muy útil para los desarrolladores de aplicaciones ya que podrían analizar cuellos de botella e identificar potenciales optimizaciones en sus aplicaciones. Los contadores de MapReduce pueden dividirse en cuatro categorías:

- **Job Counter.** Contadores de trabajos. `org.apache.hadoop.mapreduce.JobCounter`
 - **MILLIS_MAPS/MILLIS_REDUCES:** Estas métricas rastrean el tiempo en todas las tareas de *map* y *reduce*. Su utilidad completa se realiza cuando se rastrea junto con el contador de tareas `GC_TIME_MILLIS`; cuando se toman en conjunto, es posible determinar el porcentaje de tiempo gastado en la recolección de basura. Utilizando los tres contadores, podemos calcular el porcentaje de tiempo dedicado a la recolección de basura con la siguiente fórmula

```
GC_TIME_PERCENTAGE =  
    GC_TIME_MILLIS /  
    (MILLIS_MAPS + MILLIS_REDUCE)
```

Los trabajos en los que la recolección de basura consume un porcentaje significativo del tiempo de computación (por ejemplo, más del 20 por ciento) pueden beneficiarse de aumentar la cantidad de heap disponible para ellos.

- **NUM_FAILED_MAPS/NUM_FAILED_REDUCE**: Este par de métricas rastrea el número de tareas map/reduce fallidas para un trabajo. Las tareas pueden fallar por muchas razones: la salida interrumpida de la JVM, los errores en tiempo de ejecución y las tareas colgadas son algunas de las más comunes. Las tareas fallidas son toleradas hasta alcanzar el valor establecido por las siguientes propiedades:
 - `mapreduce.map.maxattempts`
 - `mapreduce.reduce.maxattempts`

Si una tarea falla el máximo de intentos, se marca como fallida.

- **DATA_LOCAL_MAPS / RACK_LOCAL_MAPS / OTHER_LOCAL_MAPS**: Estos contadores registran el número de tareas de *map* que se han realizado, agregadas por ubicación. Los datos pueden estar ubicados en el nodo que ejecuta la tarea *map*, en un nodo del mismo *rack* que el nodo que realiza la tarea de *map* o en un nodo ubicado en un rack diferente en otro lugar del clúster. Los beneficios de operar con datos locales incluyen la reducción de la sobrecarga de la red y una ejecución más rápida. Estas ventajas se pierden cuando el cálculo debe realizarse en un nodo que no está físicamente cerca de los datos que se procesan. Dado que el envío de datos a través de la red lleva tiempo, correlacionar el bajo rendimiento del trabajo con estos contadores puede ayudar a determinar por qué se ha resentido el rendimiento. Si muchas operaciones *map* tuvieran que ejecutarse en nodos donde los datos no estuvieran alojados localmente, sería razonable esperar una degradación del rendimiento.



Si muchas tareas fallan en un DataNode en particular, el problema podría estar relacionado con el hardware.



Un trabajo fallará en su totalidad si más del porcentaje de `mapreduce.map.failures.maxpercent` de las tareas *map* en el trabajo fallan o más del porcentaje de `mapreduce.reduce.maxpercent` de las tareas *reduce* fallan.

- **Tasks Counters.** Contadores de tareas. `org.apache.hadoop.mapreduce.JobCounter`
 - **REDUCE_INPUT_RECORDS:** Vigilar el número de registros de entrada para cada tarea de reducción en un trabajo sirve para identificar problemas de rendimiento relacionados con un conjunto de datos sesgados. Los datos sesgados se caracterizan por una distribución desproporcionada de los registros de entrada entre todas las operaciones *reduce* de un trabajo. Los conjuntos de datos sesgados pueden hacer que los trabajos tarden mucho más en completarse de lo esperado.
 - **SPIILLED_RECORDS:** Los datos de salida de las operaciones *map* se almacenan en memoria, pero cuando el *buffer* se llena los datos se vuelcan al disco. Un hilo separado se genera para fusionar todos los datos derramados en un único archivo ordenado más grande para las operaciones *reduce*. Los volcados al disco deben minimizarse, ya que más de un volcado provocará un aumento de la actividad en el disco, ya que el hilo de fusión tiene que leer y escribir datos en el disco. El seguimiento de esta métrica puede ayudar a determinar si es necesario realizar cambios en la configuración (como aumentar la memoria disponible para las tareas *map*) o si es necesario un procesamiento adicional (compresión) de los datos de entrada.
 - **GC_TIME_MILLIS:** Útil para determinar el porcentaje de tiempo que las tareas pasan realizando la recolección de basura.
- **Custom Counters:** MapReduce permite a los usuarios implementar contadores personalizados que son específicos para su aplicación. Los contadores personalizados se pueden utilizar para hacer un seguimiento más detallado de los recuentos. Hay un límite práctico para el número de contadores que debe utilizar, limitado por la cantidad de memoria física disponible. Hadoop tiene un valor de configuración, `job.counters.max`, que limita el número de contadores personalizados que se pueden crear; por defecto es 120.

11.3.3. YARN

Las métricas de YARN pueden agruparse en tres categorías distintas:

- **Métricas de Clúster:** Las métricas del clúster proporcionan una visión de alto nivel de la ejecución de las aplicaciones de YARN. Las métricas enumeradas aquí son agregadas a nivel de clúster.
 - **unhealthyNodes:** YARN considera que cualquier nodo con una utilización de disco que exceda el valor especificado en la configuración de YARN². Contar con el espacio suficiente en el disco es fundamental para garantizar el funcionamiento de un clúster Hadoop, y un gran

²`yarn.nodemanager.disk-health-checker.max-disk-utilization-per-disk-percentage`

número de `unhealthyNodes` debe ser rápidamente resuelto. Una vez que un nodo es marcado como `unhealthy`, otros nodos tienen que llenar el vacío, lo que puede causar una cascada de alertas a medida que un nodo tras otro se acerca a la utilización total del disco.

- **activeNodes/lostNodes:** La métrica `activeNodes` rastrea el recuento actual de nodos activos, o que funcionan normalmente. Esta métrica debería permanecer estática en ausencia de un mantenimiento preventivo. Si un gestor de nodos no mantiene el contacto con el gestor de recursos, finalmente se marcará como "perdido" y sus recursos dejarán de estar disponibles para su asignación. El umbral de tiempo de espera es configurable³, con un valor por defecto de 10 minutos (600000 ms). Los nodos pueden perder el contacto con el `ResourceManager` por varias razones, incluyendo problemas de red, fallos de hardware o un proceso colgado.
 - **appsFailed:** Si el porcentaje de tareas map o reduce fallidas supera un umbral específico⁴ la aplicación en su conjunto fallará.
 - **totalMB/allocatedMB:** Ofrecen una visión de alto nivel del uso de la memoria de tu cluster. Para aumentar la memoria disponible para el clúster, puede añadir nuevos nodos `NodeManager`, ajustar la cantidad de memoria reservada para las aplicaciones YARN o cambiar la cantidad mínima de RAM asignada a los contenedores.
- **Métricas de Aplicación:** Las métricas de aplicación proporcionan información detallada sobre la ejecución de las aplicaciones individuales de YARN. La métrica más importante de este ámbito es **progress**. El progreso proporciona una ventana en tiempo real de la ejecución de una aplicación YARN. Su valor reportado siempre estará en el rango de cero a uno (inclusive), con un valor de uno indicando una ejecución completa. Dado que la ejecución de la aplicación puede ser a menudo opaca cuando se ejecutan cientos de aplicaciones en miles de nodos, el seguimiento del progreso junto con otras métricas puede ayudar a determinar mejor la causa de cualquier degradación del rendimiento.
- **Métricas del Node Manager:** Las métricas del `NodeManager` proporcionan información sobre los recursos a nivel de nodo individual. Desde el punto de vista de YARN la métrica más importante es **containersFailed** que registra el número de contenedores que no se han podido lanzar en ese `NodeManager` en particular. La causa más común de este fallo relacionada con el hardware, por ejemplo si el disco está lleno, etc.

³a través de `yarn.resourcemanager.nm.liveness-monitor.expire-interval-ms` en `yarn-site.xml`

⁴configurado con `mapreduce.map.failures.maxpercent` y `mapreduce.reduce.failures.maxpercent`, respectivamente

Bibliografía

- [Lak14] Bhushan Lakhe. *Practical Hadoop Security*. Apress, USA, 1st edition, 2014.
- [Sin15] Gurmukh Singh. *Monitoring Hadoop*. Packt Publishing Ltd, 2015.
- [WSV14] Sameer Wadkar, Madhu Siddalingaiah, and Jason Venner. *Pro Apache Hadoop*. Springer, 2014.
- [YSC⁺18] Jianxi Yang, Chaoxiao Shen, Yaping Chi, Ping Xu, and Wei Sun. An extensible hadoop framework for monitoring performance metrics and events of openstack cloud. In *2018 IEEE 3rd International Conference on Big Data Analysis (ICBDA)*, pages 222–226. IEEE, 2018.